

Program Information

I have uploaded my C++ project to the GitHub for this class (https://github.com/tgmaxson/HPC_TGM) under folder “hw6”. This is a modified form of HW5, which now supports MPI and GPUs.

Cluster Information

I am running on Ubuntu 22.04.2 LTS via a cluster known as “Clustie” to our research group, which I have full access to. Tests are run on a test node which has nothing else running at the time of execution, except for SLURM and other background tasks which are default to Ubuntu Server installs. GCC is version 11.4.0 and ICC is version 2021.10.0 via OneAPI released on 06/09/2023. CUDA is provided via Nvidia HPC SDK version and nvcc is version cuda_12.0.r12.0/compiler.32267302_0 with CUDA version V12.0.140.

System Hardware

Motherboard: H11SSL-i CPU: Single Socket 32 Core AMD Epyc 7551P, 2.0 GHz, boost to 3.0 GHz. SMT Disabled. Memory: 256 GB of ECC DDR4-2400 via 8 x 32 GB sticks in 8 channel GPU: 2 x MSI Suprim Liquid X24G RTX 4090 with 24GB of memory.

Performance Results

```
# CPU Only
# Single Process
EXE: ./bin/hw6, RANKS: 1, SIZE: 100, GENS: 1000, SEC: 0.03000
EXE: ./bin/hw6, RANKS: 1, SIZE: 500, GENS: 1000, SEC: 0.73000
EXE: ./bin/hw6, RANKS: 1, SIZE: 1000, GENS: 1000, SEC: 3.12000
EXE: ./bin/hw6, RANKS: 1, SIZE: 2000, GENS: 1000, SEC: 11.71000
EXE: ./bin/hw6, RANKS: 1, SIZE: 4000, GENS: 1000, SEC: 47.13000
EXE: ./bin/hw6, RANKS: 1, SIZE: 8000, GENS: 1000, SEC: 202.39000
EXE: ./bin/hw6, RANKS: 1, SIZE: 12000, GENS: 1000, SEC: 480.93000

# Four Process
EXE: ./bin/hw6, RANKS: 4, SIZE: 100, GENS: 1000, SEC: 4.83000
EXE: ./bin/hw6, RANKS: 4, SIZE: 500, GENS: 1000, SEC: 4.03000
EXE: ./bin/hw6, RANKS: 4, SIZE: 1000, GENS: 1000, SEC: 4.60000
EXE: ./bin/hw6, RANKS: 4, SIZE: 2000, GENS: 1000, SEC: 5.98000
EXE: ./bin/hw6, RANKS: 4, SIZE: 4000, GENS: 1000, SEC: 17.05000
EXE: ./bin/hw6, RANKS: 4, SIZE: 8000, GENS: 1000, SEC: 57.55000
EXE: ./bin/hw6, RANKS: 4, SIZE: 12000, GENS: 1000, SEC: 123.61000
```

```
# GPU Results (GPUS=RANKS, 2 GPUs per Node)
# One Node
# Single GPU
```

```
EXE: ./bin/hw6, RANKS: 1, SIZE: 100, GENS: 1000, SEC: 0.028000
EXE: ./bin/hw6, RANKS: 1, SIZE: 500, GENS: 1000, SEC: 0.023000
EXE: ./bin/hw6, RANKS: 1, SIZE: 1000, GENS: 1000, SEC: 0.026000
EXE: ./bin/hw6, RANKS: 1, SIZE: 2000, GENS: 1000, SEC: 0.039000
EXE: ./bin/hw6, RANKS: 1, SIZE: 4000, GENS: 1000, SEC: 0.061000
EXE: ./bin/hw6, RANKS: 1, SIZE: 8000, GENS: 1000, SEC: 0.052000
EXE: ./bin/hw6, RANKS: 1, SIZE: 12000, GENS: 1000, SEC: 0.083000
EXE: ./bin/hw6, RANKS: 1, SIZE: 15000, GENS: 1000, SEC: 0.088000
EXE: ./bin/hw6, RANKS: 1, SIZE: 30000, GENS: 1000, SEC: 0.131000

# Dual GPU
EXE: ./bin/hw6, RANKS: 2, SIZE: 100, GENS: 1000, SEC: 0.042000
EXE: ./bin/hw6, RANKS: 2, SIZE: 500, GENS: 1000, SEC: 0.058000
EXE: ./bin/hw6, RANKS: 2, SIZE: 1000, GENS: 1000, SEC: 0.049000
EXE: ./bin/hw6, RANKS: 2, SIZE: 2000, GENS: 1000, SEC: 0.059000
EXE: ./bin/hw6, RANKS: 2, SIZE: 4000, GENS: 1000, SEC: 0.083000
EXE: ./bin/hw6, RANKS: 2, SIZE: 8000, GENS: 1000, SEC: 0.166000
EXE: ./bin/hw6, RANKS: 2, SIZE: 12000, GENS: 1000, SEC: 0.180000
EXE: ./bin/hw6, RANKS: 2, SIZE: 15000, GENS: 1000, SEC: 0.270000

# Two Nodes
# Quad GPU
EXE: ./bin/hw6, RANKS: 4, SIZE: 100, GENS: 1000, SEC: 5.047000
EXE: ./bin/hw6, RANKS: 4, SIZE: 500, GENS: 1000, SEC: 5.029000
EXE: ./bin/hw6, RANKS: 4, SIZE: 1000, GENS: 1000, SEC: 5.023000
EXE: ./bin/hw6, RANKS: 4, SIZE: 2000, GENS: 1000, SEC: 5.022000
EXE: ./bin/hw6, RANKS: 4, SIZE: 4000, GENS: 1000, SEC: 5.053000
EXE: ./bin/hw6, RANKS: 4, SIZE: 8000, GENS: 1000, SEC: 5.045000
EXE: ./bin/hw6, RANKS: 4, SIZE: 12000, GENS: 1000, SEC: 5.118000
EXE: ./bin/hw6, RANKS: 4, SIZE: 15000, GENS: 1000, SEC: 5.179000

# Three Nodes
# 6-GPUs
EXE: ./bin/hw6, RANKS: 6, SIZE: 100, GENS: 1000, SEC: 4.842000
EXE: ./bin/hw6, RANKS: 6, SIZE: 500, GENS: 1000, SEC: 4.833000
EXE: ./bin/hw6, RANKS: 6, SIZE: 1000, GENS: 1000, SEC: 4.837000
EXE: ./bin/hw6, RANKS: 6, SIZE: 2000, GENS: 1000, SEC: 4.821000
EXE: ./bin/hw6, RANKS: 6, SIZE: 4000, GENS: 1000, SEC: 4.816000
EXE: ./bin/hw6, RANKS: 6, SIZE: 8000, GENS: 1000, SEC: 4.852000
EXE: ./bin/hw6, RANKS: 6, SIZE: 12000, GENS: 1000, SEC: 4.878000
EXE: ./bin/hw6, RANKS: 6, SIZE: 15000, GENS: 1000, SEC: 4.877000
```

Discussion

Within any set of hardware, it can be seen that the CPU performance scales with the problem size approximately linearly as seen in HW1-4. Additionally, GPU performance is not scaling well at these problem sizes since there is not enough work to do in the game of life. I believe we are memory bound in this application and almost all performance is based on cache-hits / cache-misses. This leads to the idea to use MPI to parallelize the application over multiple GPUs.

The CPU results are accelerated by MPI as expected. This is essentially HW4 and works fine. The GPU results are not accelerated since the GPU performance for a 4000x4000 grid is approximately the same as a 2000x2000 grid, meaning using two GPUs simply adds MPI instructions without any benefit (both GPUs could do the full set of work in the same time they can do the split work almost). The non-MPI version is fastest.

Clearly the networking affects the dual node configuration, conversion of GPU data from GPU->CPU->NIC->Switch->NIC->CPU->GPU appears to take a considerable amount of time. A similar slowdown is observed in the GPU->CPU->NIC->CPU->GPU path required in the Dual GPU configuration.

To quantify these effects, the MPI timing was benchmarked by a simple timer inserted in the code around the MPI statements. This gave an ~2% time spent in MPI for the single GPU, ~35% for the dual GPU, and ~99.9% for the quad GPU and 6-GPU. These results clearly demonstrate that the cost of using MPI in this exact form gives no improvement since the communication between GPU and CPU and NIC causes more delay than simply performing twice the work on a GPU.

Solutions

While I do not pursue this options here, some solutions to this problem should be to use GPU->GPU communication and to perform multiple iterations before sending data to the CPU.

Multiple iterations can be done if we use multiple ghost-rows, but the memory cost and operations cost will increase with the number of rows. However, this will allow for less GPU<->CPU communication and speed things up overall.

On the other hand, we could simply avoid GPU->GPU communication but this requires a supported platform. On this cluster, there is no NVLink between GPUs as 4090s do not support this feature and the only cluster that I have any access to with an NVLink Switch is NERSC I believe.